Aperture — Technical Architecture (Part 2)

Version: v1.0

Document Type: Technical Architecture

Related Documents

- Product Requirements Document
- Development Roadmap
- System Architecture
- Low-Level Design

---

7. Data Architecture Philosophy

Data is Aperture's most valuable asset.

Unlike traditional movie applications that simply proxy data from TMDb, Aperture maintains its own canonical data model. External providers are treated as **data sources**, not the application's source of truth.

This architectural decision enables:

- provider independence
- metadata enrichment
- AI-generated insights
- consistent relationships
- future support for additional metadata providers

The long-term objective is to make Aperture's internal dataset richer than any individual upstream source.

---

8. Primary Database

Decision

PostgreSQL

Problem

The application stores highly relational data:

- users
- movies
- TV series
- seasons
- episodes
- ratings
- reviews
- follows
- lists
- comments
- recommendations

Maintaining strong consistency between these entities is essential.

Alternatives Considered

MongoDB

Advantages

- Flexible schema
- Rapid prototyping
- Horizontal scaling

Disadvantages

- Complex joins
- Data duplication
- Difficult relationship modeling

MongoDB is well suited to document-oriented workloads, but Aperture revolves around interconnected relationships.

MySQL

Reliable and familiar, but PostgreSQL provides richer functionality that benefits this project, including advanced indexing, JSON support, extensions such as pgvector, and superior full-text capabilities.

DynamoDB

Excellent for predictable access patterns at hyperscale, but it introduces significant modeling complexity that is unnecessary for the current scale and learning objectives.

Decision

PostgreSQL provides the best balance of:

- relational integrity
- mature ecosystem
- extensibility
- developer experience
- production adoption

Future Evolution

As scale increases:

- read replicas
- connection pooling
- partitioning
- archival tables

can be introduced without changing the application's programming model.

Interview Discussion

Why PostgreSQL instead of MongoDB?

Because Aperture models relationships more often than documents. Reviews, users, follows, lists, ratings, cast members, and recommendations all benefit from relational consistency and transactional guarantees.

---

9. ORM

Decision

SQLAlchemy

Why?

Requirements:

- expressive models
- migrations
- relationship management
- transaction support

SQLAlchemy provides a mature ORM while still allowing direct SQL when performance optimization becomes necessary.

Avoid hiding database behavior behind excessive abstraction.

---

10. Database Migration Strategy

Decision

Alembic

Every schema modification should be versioned.

Benefits:

- reproducible deployments
- rollback capability
- collaborative development
- deployment automation

Schema changes should never be performed manually in production.

---

11. Caching Strategy

Decision

Redis

Caching is introduced because repeated database access eventually becomes more expensive than memory access.

Redis is used selectively rather than universally.

Initial Cache Candidates

- popular movies
- trending content
- homepage sections
- search suggestions
- TMDb metadata
- user sessions
- rate limiting counters

What Should Not Be Cached

Avoid caching:

- highly personalized responses
- rapidly changing data
- transactional writes

unless profiling demonstrates clear value.

Cache Invalidation

Cache invalidation follows simple rules:

- update cache after writes
- expire stale metadata automatically
- invalidate only affected keys

Avoid global cache flushes.

Future Evolution

Introduce:

- distributed Redis
- cache warming
- write-through strategies
- background refresh

only after traffic justifies the additional complexity.

Interview Discussion

Why Redis instead of Memcached?

Redis offers richer data structures, persistence options, atomic operations, rate limiting primitives, and future extensibility beyond simple key-value caching.

---

12. Search Architecture

Search is intentionally separated from transactional storage.

The database answers:

"What is true?"

The search engine answers:

"What is relevant?"

Phase 1

PostgreSQL full-text search

Simple

Reliable

Minimal operational overhead

Phase 2

OpenSearch

Requirements:

- autocomplete
- fuzzy matching
- typo tolerance
- weighted ranking
- faceted filtering

Why Not OpenSearch Immediately?

Operating another distributed system from day one increases maintenance burden.

PostgreSQL is sufficient until search requirements exceed its capabilities.

This aligns with Aperture's principle of evolutionary architecture.

Future Enhancements

- synonym dictionaries
- search analytics
- personalized ranking
- semantic reranking

---

13. Metadata Ingestion

Metadata enters Aperture through a dedicated ingestion pipeline.

TMDb remains the authoritative external provider.

However, all incoming data is transformed into Aperture's internal schema.

Pipeline:

TMDb

↓

API Client

↓

Validation

↓

Normalization

↓

Deduplication

↓

Persistence

↓

Indexing

↓

Availability to users

Benefits

- provider independence
- cleaner database
- AI enrichment
- easier testing
- future extensibility

---

14. Streaming Availability

Streaming availability is treated as metadata rather than business logic.

Responsibilities include:

- provider mapping
- region awareness
- subscription availability
- rental availability
- purchase availability

Because availability changes frequently, synchronization should occur independently from static movie metadata.

---

15. Recommendation Architecture

Recommendations are introduced gradually.

Phase 1

Metadata similarity

Signals include:

- genres
- cast
- director
- keywords

Simple and deterministic.

Phase 2

Behavioral recommendations

Signals include:

- ratings
- reviews
- watchlists
- viewing history
- favorites

Phase 3

Hybrid recommendations

Combine:

- metadata
- collaborative filtering
- embeddings
- popularity
- recency

The final recommendation score is generated from multiple weighted signals rather than any single algorithm.

Design Principle

Recommendation quality should improve over time without changing the public API.

Clients request recommendations.

The backend decides how they are generated.

---

16. Vector Search Strategy

Semantic understanding becomes important only after traditional recommendation quality has plateaued.

Decision

pgvector

Why?

Advantages:

- integrates directly with PostgreSQL
- simple operational model
- ideal for early-stage products
- minimizes infrastructure

Alternatives

Dedicated vector databases:

- Pinecone
- Weaviate
- Milvus

These become attractive only when embedding scale or latency requirements exceed PostgreSQL's capabilities.

Future Evolution

Migration should be invisible to consumers.

Embedding generation remains independent of vector storage.

---

17. AI Integration Strategy

Artificial Intelligence should enhance—not replace—the core platform.

The application must remain fully functional without AI.

AI should improve:

- recommendations
- discovery
- search
- personalization
- editorial experiences

This philosophy minimizes operational risk while allowing rapid experimentation.

---

18. Background Processing

Expensive or non-interactive work should execute asynchronously.

Examples:

- metadata synchronization
- recommendation generation
- search indexing
- embedding creation
- email delivery
- notification fan-out

The user should never wait for work that does not directly affect the immediate response.

Future Queue Evolution

Phase 1

Simple background tasks

↓

Phase 2

Redis-backed queue

↓

Phase 3

Dedicated message broker if justified

The architecture deliberately postpones distributed messaging until operational complexity is warranted.

---

19. API Design Philosophy

The API should be:

- resource-oriented
- predictable
- versioned
- well documented
- backward compatible where practical

Guiding principles:

- consistent naming
- meaningful status codes
- structured error responses
- pagination by default
- idempotent operations where appropriate

Automatic OpenAPI documentation should remain synchronized with implementation.

---

20. Summary

The technical architecture prioritizes technologies that maximize engineering clarity, long-term maintainability, and production readiness while avoiding unnecessary complexity.

Every technology introduced into Aperture exists because it solves a concrete engineering problem. Where possible, simpler solutions are adopted first, with clear migration paths documented for future growth.

This approach ensures that the platform remains approachable for a single developer today while providing a credible evolution path toward a large-scale production system.